

AL Test Generator

Manual de Usuario

Versión 1.0

Agente inteligente para la generación automática de
codeunits de prueba en AL para
Microsoft Dynamics 365 Business Central

Trabajo de Fin de Grado	Ingeniería Informática
Institución	Universitat Jaume I
Fecha	Mayo de 2026

Índice

1. Introducción	2
2. Público al que va dirigido	2
3. Instalación y configuración	2
3.1. Requisitos previos	2
3.2. Obtención del proyecto	3
3.3. Entorno virtual e instalación de dependencias	3
3.4. Configuración del archivo .env	3
3.5. Integración con el proyecto AL	3
3.6. Construcción de la base de conocimiento	4
4. Guía de uso	5
4.1. generate_al_test — Generación de tests	5
4.2. generate_al_test_from_recording — Tests desde grabación	6
4.3. search_al_knowledge — Consulta a la base de conocimiento	6
4.4. refresh_examples y rebuild_vectorstore — Mantenimiento	6
4.5. Consideraciones sobre el código generado	7

1. Introducción

AL Test Generator es un agente de inteligencia artificial diseñado para generar automáticamente codeunits de prueba en el lenguaje AL (*Application Language*) de BC. A partir de una descripción en lenguaje natural, el sistema produce código de prueba compilable listo para integrarse en el proyecto, reduciendo significativamente el tiempo dedicado a la escritura manual de tests.

El agente combina tres capacidades complementarias:

- **Recuperación de contexto del proyecto.** Antes de generar código, el agente lee los ficheros .al del proyecto activo para descubrir los objetos, procedimientos y dependencias reales, evitando la invención de identificadores.
- **Base de conocimiento especializada.** Un índice vectorial construido a partir de un libro de buenas prácticas AL y de tests de referencia curados proporciona patrones probados que el agente utiliza como guía.
- **Validación automática.** El código generado se compila con `alc.exe` antes de entregarse. Si aparecen errores, el agente realiza hasta dos rondas de corrección automática.

El sistema se expone como un servidor **MCP** (*Model Context Protocol*) y se consume desde el chat de **GitHub Copilot** en modo agente dentro de Visual Studio Code.

Este sistema es el resultado de un Trabajo de Fin de Grado y está orientado a entornos de desarrollo. El código generado debe ser revisado por el desarrollador antes de su incorporación al repositorio.

2. Público al que va dirigido

Este manual está destinado a **desarrolladores de BC** que cumplan los siguientes perfiles:

- Conocimiento del lenguaje AL y de la estructura de proyectos de extensiones de BC.
- Familiaridad con Visual Studio Code como entorno de desarrollo.
- Acceso a una suscripción activa de **GitHub Copilot** (el plan gratuito es suficiente si se proporciona la misma API usada para el agente en la extensión).

No se requiere experiencia previa en inteligencia artificial, grandes modelos de lenguaje ni sistemas de recuperación de información. El presente manual proporciona toda la información necesaria para instalar, configurar y operar la herramienta.

El sistema resulta especialmente adecuado para los siguientes casos de uso:

- Equipos que desean incorporar la práctica de *testing* automatizado en proyectos AL sin dedicar un alto coste inicial en escritura de tests.
- Desarrolladores que necesitan generar una batería inicial de tests para código no cubierto.
- Escenarios en los que se dispone de grabaciones de sesiones de BC y se desea convertirlas automáticamente en tests funcionales.

3. Instalación y configuración

3.1. Requisitos previos

Además, se necesita al menos una **clave de API** de proveedor LLM.

Componente	Versión mínima	Observaciones
Python	3.10	Requerido por las dependencias de LangChain
Git	—	Para la clonación del repositorio
Visual Studio Code	1.90+	Con la extensión GitHub Copilot instalada
GitHub Copilot	Plan gratuito	Necesario para el modo agente con MCP. Puede sustituirse registrando la clave de API del LLM como proveedor de lenguaje en VS Code.

Cuadro 1: Requisitos de software para la ejecución del sistema

3.2. Obtención del proyecto

Descomprima el ZIP en el directorio de trabajo deseado.

3.3. Entorno virtual e instalación de dependencias

```
python -m venv .venv

# Windows
.venv\Scripts\activate

# macOS / Linux
source .venv/bin/activate

pip install -r requirements.txt
```

El primer arranque descarga el modelo de *embeddings* BAAI/bge-m3 (570 MB) desde HuggingFace y lo almacena en caché local. Las ejecuciones posteriores no requieren conexión para este componente.

3.4. Configuración del archivo .env

Copie el archivo de ejemplo y complete las variables necesarias:

```
# Windows
copy .env.example .env

# macOS / Linux
cp .env.example .env
```

Edite el fichero .env y defina al menos:

```
ANTHROPIC_API_KEY=sk-ant-...

# Ruta absoluta al directorio del proyecto AL (donde esta app.json)
PROJECT_ROOT=C:\ruta\a\MiProyectoBC
```

3.5. Integración con el proyecto AL

El proyecto AL que se desea testear necesita dos ficheros de configuración incluidos en la carpeta `project-template/`:

```
project-template/  
  .github/  
    copilot-instructions.md    <- reglas de comportamiento para  
      Copilot  
  .vscode/  
    mcp.json                  <- configuracion del servidor MCP
```

Paso 1. Copie ambas carpetas a la raíz del proyecto AL.

Paso 2. Abra `.vscode/mcp.json` y sustituya las rutas de ejemplo por las rutas absolutas reales de su máquina:

Listing 1: mcp.json – Windows

```
{  
  "servers": {  
    "al-test-generator": {  
      "type": "stdio",  
      "command": "C:\\ruta\\a\\TFGLaberit\\.venv\\Scripts\\python.  
        exe",  
      "args":    ["C:\\ruta\\a\\TFGLaberit\\mcp_server.py"],  
      "env": {  
        "PROJECT_ROOT": "${workspaceFolder}"  
      }  
    }  
  }  
}
```

Listing 2: mcp.json – macOS / Linux

```
{  
  "servers": {  
    "al-test-generator": {  
      "type": "stdio",  
      "command": "/ruta/a/TFGLaberit/.venv/bin/python",  
      "args":    ["/ruta/a/TFGLaberit/mcp_server.py"],  
      "env": {  
        "PROJECT_ROOT": "${workspaceFolder}"  
      }  
    }  
  }  
}
```

El campo `command` debe apuntar al ejecutable de Python **dentro del entorno virtual** (`.venv`), no al Python del sistema. De lo contrario, el servidor arrancará sin las dependencias instaladas.

Paso 3. Abra VS Code en la carpeta del proyecto AL, active el modo agente en el chat de Copilot (`Ctrl+Alt+I` → selector de modo **Agent**) y verifique que `al-test-generator` aparece en el icono de herramientas. Para no tener que adquirir la suscripción de Github Copilot, es necesario añadir la API del LLM como agente global en la extensión.

3.6. Construcción de la base de conocimiento

La base de datos vectorial debe construirse una única vez por máquina. Ejecute desde el chat de Copilot en modo agente:

```
#rebuild_vectorstore
```

Este proceso indexa el libro de buenas prácticas AL y los tests de referencia. El resultado se persiste en `chroma_al_testing/` y se carga automáticamente en arranques posteriores.

La primera ejecución puede requerir entre 5 y 15 minutos (descarga del modelo de *embeddings*). No cierre VS Code durante el proceso.

4. Guía de uso

Todas las herramientas se invocan desde el chat de GitHub Copilot en modo agente con el prefijo `@al-test-generator`:

```
@al-test-generator <descripcion o nombre de herramienta>
```

4.1. generate_al_test — Generación de tests

Genera un codeunit de tests AL completo a partir de una descripción en lenguaje natural. Es la herramienta principal del sistema.

```
@al-test-generator Genera un test para el procedimiento
PostSalesOrder en el codeunit SalesManagement que verifique que
se contabiliza correctamente una orden con un descuento del 10%
```

Internamente, el agente ejecuta tres fases antes de entregar el código:

1. **Descubrimiento.** Lee los ficheros `.al` del proyecto para identificar los objetos y procedimientos relevantes.
2. **Generación.** Combina el contexto del proyecto con los patrones recuperados de la base de conocimiento y produce el codeunit con estructura `GIVEN / WHEN / THEN`.
3. **Compilación.** Verifica el código con `alc.exe` y aplica hasta dos rondas de corrección automática si se detectan errores.

Ejemplo de salida:

```
1 codeunit 50100 "SalesManagement Tests"
2 {
3     Subtype = Test;
4     TestPermissions = Disabled;
5
6     trigger OnRun() begin end;
7
8     [Test]
9     procedure PostSalesOrderWithDiscountUpdatesLedger()
10    var
11        SalesHeader: Record "Sales Header";
12        SalesLine: Record "Sales Line";
13        PostedDocNo: Code[20];
14    begin
15        // [GIVEN] Una orden de venta con descuento del 10%
16        Initialize();
17        LibrarySales.CreateSalesOrder(SalesHeader);
18        LibrarySales.CreateSalesLineWithDiscount(SalesLine,
19            SalesHeader, 10);
20
21        // [WHEN] Se contabiliza la orden
22        PostedDocNo := LibrarySales.PostSalesDocument(SalesHeader,
23            true, true);
```

```

22
23         // [THEN] El movimiento contable refleja el descuento
24         VerifyPostedSalesInvoiceDiscount(PostedDocNo, 10);
25     end;
26
27     local procedure Initialize()
28     begin
29         if IsInitialized then exit;
30         IsInitialized := true;
31         Commit();
32     end;
33
34     var
35         LibrarySales:           Codeunit "Library - Sales";
36         IsInitialized:          Boolean;
37 }

```

Listing 3: Codeunit de tests generado automáticamente

Cuanto más específica sea la consulta —nombre exacto del procedimiento, codeunit a la que pertenece, comportamiento a verificar— mejores resultados se obtienen. Abrir el fichero fuente relevante en el editor antes de preguntar también ayuda al agente a descubrir los identificadores correctos.

4.2. `generate_al_test_from_recording` — Tests desde grabación

Genera un codeunit de tests a partir de una **grabación de sesión de Business Central** exportada en formato YAML. Permite convertir directamente una secuencia de pasos registrados en la interfaz de BC en código de prueba funcional.

```
@al-test-generator Genera un test a partir de la grabacion en C:\
grabaciones\crear_cliente.yaml
```

4.3. `search_al_knowledge` — Consulta a la base de conocimiento

Busca en el índice vectorial (libro de buenas prácticas + tests curados) sin generar código. Útil para resolver dudas conceptuales o localizar patrones.

```
#search_al_knowledge Busca en la base de conocimiento como
implementar correctamente el metodo Initialize en un codeunit de
tests
```

4.4. `refresh_examples` y `rebuild_vectorstore` — Mantenimiento

Herramienta	Cuándo usarla	Duración
<code>refresh_examples</code>	Al añadir o modificar ficheros en <code>Data/TestExamples/</code> . Reindexación incremental (solo procesa los ficheros nuevos).	Rápida
<code>rebuild_vectorstore</code>	Primera instalación, actualización del libro de buenas prácticas o reconstrucción completa del índice.	5–15 min

Cuadro 2: Herramientas de mantenimiento de la base de conocimiento

```
#refresh_examples  
#rebuild_vectorstore
```

4.5. Consideraciones sobre el código generado

El código producido por el agente debe ser revisado por el desarrollador antes de incorporarlo al repositorio. La corrección sintáctica (compilación) es verificada automáticamente; la corrección semántica —que el test efectivamente prueba lo que se pretende— es responsabilidad del desarrollador.

Se recomienda comprobar en particular que:

- Todos los identificadores (tablas, codeunits, campos, procedimientos) existen realmente en el proyecto.
- La estructura GIVEN/WHEN/THEN refleja la intención del test.
- El método `Initialize()` limpia el estado de la base de datos antes de cada test.
- Los manejadores de mensajes (`MessageHandler`, `ModalPageHandler`) están declarados si el código bajo prueba los requiere.